

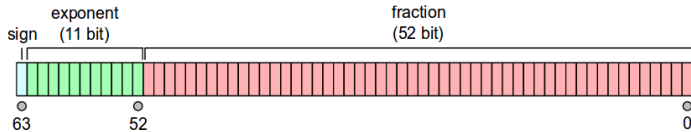
Algoritmos numéricos y propagación del error

Programación III - UNGS



Repaso: Punto flotante

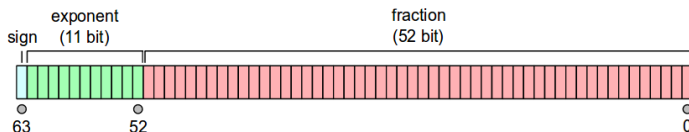
- Representamos **números reales** en Java por medio de **float** y **double**.





Repaso: Punto flotante

- Representamos **números reales** en Java por medio de **float** y **double**.

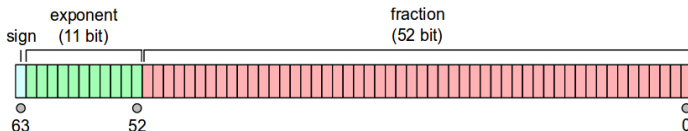


- La **representación de punto flotante** escribe un número $a = (-1)^s 0.a_1a_2 \dots a_n \times 2^{e_1 \dots e_k}$.



Repaso: Punto flotante

- Representamos **números reales** en Java por medio de **float** y **double**.

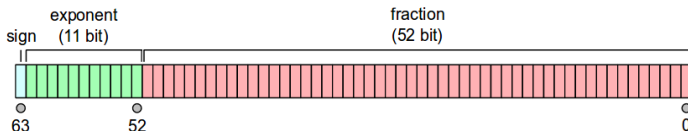


- La **representación de punto flotante** escribe un número $a = (-1)^s 0.a_1a_2 \dots a_n \times 2^{e_1 \dots e_k}$.
- El bit s es el **signo** de a .



Repaso: Punto flotante

- Representamos **números reales** en Java por medio de **float** y **double**.

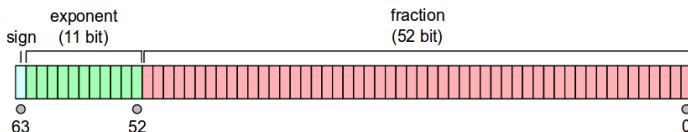


- La **representación de punto flotante** escribe un número $a = (-1)^s 0.a_1a_2 \dots a_n \times 2^{e_1 \dots e_k}$.
 - El bit s es el **signo** de a .
 - La secuencia $a_1, \dots, a_n$ es la **mantisa**, y tiene n bits.



Repaso: Punto flotante

- Representamos **números reales** en Java por medio de **float** y **double**.



- La **representación de punto flotante** escribe un número $a = (-1)^s 0.a_1a_2 \dots a_n \times 2^{e_1 \dots e_k}$.
 - El bit s es el **signo** de a .
 - La secuencia $a_1, \dots, a_n$ es la **mantisa**, y tiene n bits.
 - La secuencia $e_1, \dots, e_k$ es el **exponente**, y tiene k bits.

Punto flotante

- ¿Por qué usar **float** y **double** teniendo disponible **BigDecimal**?

Punto flotante

- ¿Por qué usar **float** y **double** teniendo disponible **BigDecimal**?
 1. No siempre conocemos el **rango** de los valores con los cuales vamos a operar.

Punto flotante

- ¿Por qué usar **float** y **double** teniendo disponible **BigDecimal**?
 1. No siempre conocemos el **rango** de los valores con los cuales vamos a operar.
 2. La implementación de **BigDecimal** recurre a una emulación de la aritmética de punto fijo, y entonces es muy lenta.

Punto flotante

- ¿Por qué usar **float** y **double** teniendo disponible **BigDecimal**?
 1. No siempre conocemos el **rango** de los valores con los cuales vamos a operar.
 2. La implementación de **BigDecimal** recurre a una emulación de la aritmética de punto fijo, y entonces es muy lenta.
- Los tipos **float** y **double** se utilizan para algoritmos intensivos en cálculos no monetarios.

Punto flotante

- ¿Por qué usar **float** y **double** teniendo disponible **BigDecimal**?
 1. No siempre conocemos el **rango** de los valores con los cuales vamos a operar.
 2. La implementación de **BigDecimal** recurre a una emulación de la aritmética de punto fijo, y entonces es muy lenta.
- Los tipos **float** y **double** se utilizan para algoritmos intensivos en cálculos no monetarios.
- Es relevante estudiar el comportamiento de los algoritmos que operan con estos tipos de datos, para tener resultados tan **precisos** como sea posible.

Punto flotante

- Sea x el valor que queremos calcular o representar, y sea $\hat{x}$ el valor realmente obtenido en una variable de tipo **float** o **double**.

Punto flotante

- Sea x el valor que queremos calcular o representar, y sea $\hat{x}$ el valor realmente obtenido en una variable de tipo **float** o **double**.
 1. El **error absoluto** de la representación es $|x - \hat{x}|$.

Punto flotante

- Sea x el valor que queremos calcular o representar, y sea $\hat{x}$ el valor realmente obtenido en una variable de tipo **float** o **double**.
 1. El **error absoluto** de la representación es $|x - \hat{x}|$.
 2. El **error relativo** de la representación es $|\frac{x - \hat{x}}{x}|$, si $x \neq 0$.

Punto flotante

- Sea x el valor que queremos calcular o representar, y sea $\hat{x}$ el valor realmente obtenido en una variable de tipo **float** o **double**.
 1. El **error absoluto** de la representación es $|x - \hat{x}|$.
 2. El **error relativo** de la representación es $|\frac{x - \hat{x}}{x}|$, si $x \neq 0$.
- Se utiliza habitualmente el error relativo (salvo que $x \cong 0$), porque proporciona una mejor medida del error obtenido.

Problemas y algoritmos numéricos

- Un **problema numérico** se puede ver como una función $f : D_1 \times \cdots \times D_n \rightarrow C_1 \times \cdots \times C_m$, donde al menos un D_i y al menos un C_j son $\mathbb{R}$.

Problemas y algoritmos numéricos

- Un **problema numérico** se puede ver como una función $f : D_1 \times \cdots \times D_n \rightarrow C_1 \times \cdots \times C_m$, donde al menos un D_i y al menos un C_j son $\mathbb{R}$.
- Un **algoritmo numérico** para el problema f es un procedimiento que calcula una aproximación de $f(x)$.

Problemas y algoritmos numéricos

- Un **problema numérico** se puede ver como una función $f : D_1 \times \cdots \times D_n \rightarrow C_1 \times \cdots \times C_m$, donde al menos un D_i y al menos un C_j son $\mathbb{R}$.
- Un **algoritmo numérico** para el problema f es un procedimiento que calcula una aproximación de $f(x)$.
 1. Un algoritmo numérico puede no ser un procedimiento finito. Se admiten procedimientos infinitos, que **convergen** al valor buscado.

Problemas y algoritmos numéricos

- Un **problema numérico** se puede ver como una función $f : D_1 \times \cdots \times D_n \rightarrow C_1 \times \cdots \times C_m$, donde al menos un D_i y al menos un C_j son $\mathbb{R}$.
- Un **algoritmo numérico** para el problema f es un procedimiento que calcula una aproximación de $f(x)$.
 1. Un algoritmo numérico puede no ser un procedimiento finito. Se admiten procedimientos infinitos, que **convergen** al valor buscado.
 2. En este caso, se interrumpe el procedimiento luego de una cantidad de iteraciones, y se tiene un **error de convergencia** asociado con las iteraciones faltantes.

Problemas y algoritmos numéricos

- Un problema numérico se dice **bien condicionado** si $x \cong x'$ implica $f(x) \cong f(x')$. En caso contrario, se dice **mal condicionado**.

Problemas y algoritmos numéricos

- Un problema numérico se dice **bien condicionado** si $x \cong x'$ implica $f(x) \cong f(x')$. En caso contrario, se dice **mal condicionado**.
- Un algoritmo numérico se dice **estable** si $F(x) \cong f(x')$ para todo $x' \cong x$.

Problemas y algoritmos numéricos

- Un problema numérico se dice **bien condicionado** si $x \cong x'$ implica $f(x) \cong f(x')$. En caso contrario, se dice **mal condicionado**.
- Un algoritmo numérico se dice **estable** si $F(x) \cong f(x')$ para todo $x' \cong x$.
- Algunas operaciones son muy poco estables:

Problemas y algoritmos numéricos

- Un problema numérico se dice **bien condicionado** si $x \cong x'$ implica $f(x) \cong f(x')$. En caso contrario, se dice **mal condicionado**.
- Un algoritmo numérico se dice **estable** si $F(x) \cong f(x')$ para todo $x' \cong x$.
- Algunas operaciones son muy poco estables:
 1. La resta de dos números parecidos y de igual signo puede generar un resultado con error relativo muy alto. Este efecto se conoce como **cancelación catastrófica**.

Problemas y algoritmos numéricos

- Un problema numérico se dice **bien condicionado** si $x \cong x'$ implica $f(x) \cong f(x')$. En caso contrario, se dice **mal condicionado**.
- Un algoritmo numérico se dice **estable** si $F(x) \cong f(x')$ para todo $x' \cong x$.
- Algunas operaciones son muy poco estables:
 1. La resta de dos números parecidos y de igual signo puede generar un resultado con error relativo muy alto. Este efecto se conoce como **cancelación catastrófica**.
 2. Dividir por un número muy cercano a cero también puede generar un error relativo alto.

Problemas y algoritmos numéricos

- **Ejemplo.** Dados $a, b, c \in \mathbb{R}$, encontrar las raíces de $ax^2 + bx + c = 0$.

Problemas y algoritmos numéricos

- **Ejemplo.** Dados $a, b, c \in \mathbb{R}$, encontrar las raíces de $ax^2 + bx + c = 0$.
- Si x_1 y x_2 son las raíces, este problema está **mal condicionado** cuando $x_1 \cong 0$ y $|x_2| \gg 0$.

Problemas y algoritmos numéricos

- **Ejemplo.** Dados $a, b, c \in \mathbb{R}$, encontrar las raíces de $ax^2 + bx + c = 0$.
- Si x_1 y x_2 son las raíces, este problema está **mal condicionado** cuando $x_1 \cong 0$ y $|x_2| \gg 0$.
- Consideremos el algoritmo clásico:

$$x_1 = \frac{-b - \sqrt{b^2 - 4ac}}{2a}, \quad x_2 = \frac{-b + \sqrt{b^2 - 4ac}}{2a}.$$

Problemas y algoritmos numéricos

- **Ejemplo.** Dados $a, b, c \in \mathbb{R}$, encontrar las raíces de $ax^2 + bx + c = 0$.
- Si x_1 y x_2 son las raíces, este problema está **mal condicionado** cuando $x_1 \cong 0$ y $|x_2| \gg 0$.
- Consideremos el algoritmo clásico:

$$x_1 = \frac{-b - \sqrt{b^2 - 4ac}}{2a}, \quad x_2 = \frac{-b + \sqrt{b^2 - 4ac}}{2a}.$$

- Dependiendo de los datos de entrada, este algoritmo puede dar resultados con muy poca precisión!

Problemas y algoritmos numéricos

●

```

1  double a = 1;
2  double b = -300000000;
3  double c = 2;
4
5  double r1 = (-b - Math.sqrt(b*b - 4*a*c)) / (2*a);
6  double r2 = (-b + Math.sqrt(b*b - 4*a*c)) / (2*a);

```

Problemas y algoritmos numéricos

- ---

```

1  double a = 1;
2  double b = -30000000;
3  double c = 2;
4
5  double r1 = (-b - Math.sqrt(b*b - 4*a*c)) / (2*a);
6  double r2 = (-b + Math.sqrt(b*b - 4*a*c)) / (2*a);
    
```

- Las raíces son $r_1 = 2/3 \times 10^{-7}$ y $r_2 = 10^7$.

Problemas y algoritmos numéricos

```

1  double a = 1;
2  double b = -300000000;
3  double c = 2;
4
5  double r1 = (-b - Math.sqrt(b*b - 4*a*c)) / (2*a);
6  double r2 = (-b + Math.sqrt(b*b - 4*a*c)) / (2*a);

```

- Las raíces son $r_1 = 2/3 \times 10^{-7}$ y $r_2 = 10^7$.
- Obtenemos $r_1 = 2.002343535 \times 10^{-7}$ y $r_2 = 9999999.99$.

Problemas y algoritmos numéricos

```

1  double a = 1;
2  double b = -300000000;
3  double c = 2;
4
5  double r1 = (-b - Math.sqrt(b*b - 4*a*c)) / (2*a);
6  double r2 = (-b + Math.sqrt(b*b - 4*a*c)) / (2*a);

```

- Las raíces son $r_1 = 2/3 \times 10^{-7}$ y $r_2 = 10^7$.
- Obtenemos $r_1 = 2.002343535 \times 10^{-7}$ y $r_2 = 9999999.99$.
- El error relativo de r_1 es del **200%**!

Problemas y algoritmos numéricos

- Un **algoritmo estable** en este caso es calcular r_2 con la fórmula cuadrática y después obtener $x_1 = 2a/r_2$.

Problemas y algoritmos numéricos

- Un **algoritmo estable** en este caso es calcular r_2 con la fórmula cuadrática y después obtener $x_1 = 2a/r_2$.

```

1  double a = 1;
2  double b = -1000000.001;
3  double c = 10000;
4
5  double r2 = (-b + Math.sqrt(b*b - 4*a*c)) / (2*a);
6  double r1 = 2*a/r2;

```

Problemas y algoritmos numéricos

- Un **algoritmo estable** en este caso es calcular r_2 con la fórmula cuadrática y después obtener $x_1 = 2a/r_2$.

```

1  double a = 1;
2  double b = -1000000.001;
3  double c = 10000;
4
5  double r2 = (-b + Math.sqrt(b*b - 4*a*c)) / (2*a);
6  double r1 = 2*a/r2;

```

- Muchas veces, se obtiene un algoritmo estable reescribiendo los cálculos del algoritmo original.

Problemas y algoritmos numéricos

- **Ejemplo.** Dado $x \in \mathbb{R}$, calcular $f(x) = e^x$ por medio de la **serie de Taylor**:

$$e^x = \sum_{n=0}^{\infty} \frac{x^n}{n!}.$$

Problemas y algoritmos numéricos

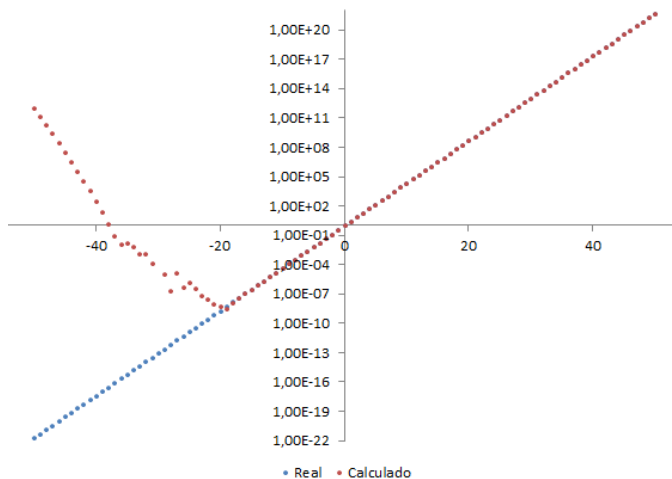
- **Ejemplo.** Dado $x \in \mathbb{R}$, calcular $f(x) = e^x$ por medio de la **serie de Taylor**:

$$e^x = \sum_{n=0}^{\infty} \frac{x^n}{n!}.$$

```

1  public static double exp(double x, int terminos)
2  {
3      double pot = 1, fact = 1, ret = 0;
4      for(int i=1; i<terminos; ++i)
5      {
6          ret += pot / fact;
7          pot *= x;
8          fact *= i;
9      }
10
11     return ret;
12 }
```

Problemas y algoritmos numéricos



Problemas y algoritmos numéricos

- Los resultados son muy imprecisos cuando $x < 0$.

Problemas y algoritmos numéricos

- Los resultados son muy imprecisos cuando $x < 0$.
- Esto se debe a que términos consecutivos de la serie tienen signos opuestos, y se produce cancelación catastrófica entre ellos:

$$e^{-2} = 1 - \frac{2}{1!} + \frac{4}{2!} - \frac{8}{3!} + \frac{16}{4!} - \frac{32}{5!} + \dots$$

Problemas y algoritmos numéricos

- Los resultados son muy imprecisos cuando $x < 0$.
- Esto se debe a que términos consecutivos de la serie tienen signos opuestos, y se produce cancelación catastrófica entre ellos:

$$e^{-2} = 1 - \frac{2}{1!} + \frac{4}{2!} - \frac{8}{3!} + \frac{16}{4!} - \frac{32}{5!} + \dots$$

- En este caso, un algoritmo estable consiste en calcular $z = e^{|x|}$ y, si $x < 0$, retornar $1/z$.